

If you run the sketch at this point, you'll see a series of curves alright, but their shape and color will remain uniform throughout. We haven't yet incremented our noise coordinate value – a vital step, but one all too easy to forget (I've omitted this line numerous times myself, leading to many a head-scratching moment). We'll take care of this step next and also increase the hue while we're at it, modulating it slightly via the `noiseValue` variable. As a final flourish, we'll throw in some randomized “bubbles” (see Figure 6-12). Don't ask me why.

```
// Increment the noise and hue.
n += speed;
hue = (hue % 360) + (noiseValue / 25);

// 10% chance of spawning a 'bubble'.
if (Gen.chance(10)) {
  svg.create('circle').set({
    r: Gen.random(5, 50),
    cx: Gen.random(150, 850),
    cy: Gen.random(150, 850),
    fill: `hsl(0 0% 100% / 0.1)`,
    stroke: '#888'
  });
}
```

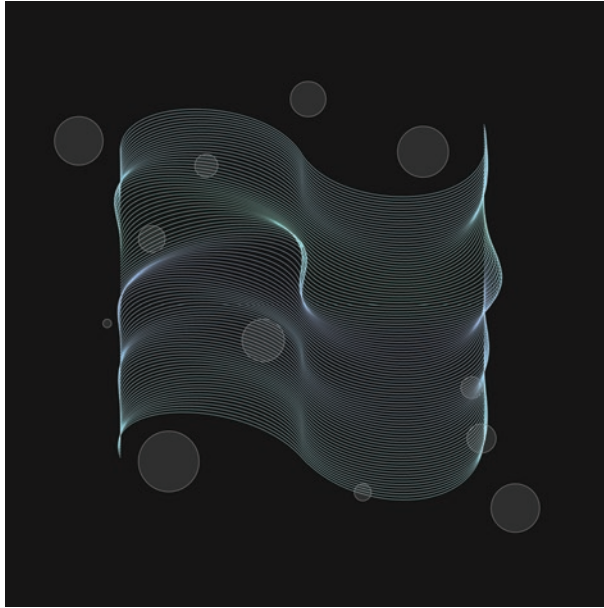


Figure 6-12. *Noise-modulated organic curves*

An Easier Option

Having to juggle six coordinates for every curve segment (or four when we use the `S` command) can get a little trying at times. Wouldn't it be nice to be able to connect a series of single points with a smooth curve, without having to worry about all those extra control points?

Glad you asked! SvJs comes equipped with a method called `createCurve()` that can do just this. Under the hood, it harnesses the power of the cubic Bezier curve but abstracts away the control points so that constructing a curve is a less intimidating affair. This does sacrifice a large degree of control, but in some cases, we may not need the level of fine-tuning that the manual curve commands offer. With the `createCurve()` method, all we need to worry about are the points.

```
// The syntax of the createCurve() method.  
createCurve(pointsArray, [curveFactor]);
```

In crude terms, we can think of this as a join-the-dots method, the main difference being that we need to supply the dots rather than do the joining. The first argument, `pointsArray`, is, as the name suggests, an array of `[x, y]` coordinate pairs. It can be supplied as either a standard one-dimensional array or as a two-dimensional array (i.e., an array of arrays).

```
// A one-dimensional array of points.
let points1D = [25, 30, 45, 20, 80, 10, 90, 40];

// A two-dimensional array of the same points.
let points2D = [
  [25, 30], [45, 20], [80, 10], [90, 40]
];
```

The `curveFactor` argument allows us to modify the tension between the points. At a value of 0, the points are connected by a straight line. At a value of 1 (the default), the tension is looser and the curve much smoother. In general, the most usable results lie in the range of 0 to 2; stray too far beyond that and the underlying control points will overlap and create some truly chaotic squiggles. On certain occasions though, this may be what you want.

In Figure 6-13, we can see the aforementioned `curveFactor` values in action. The uppermost curve is set to 0, the middle curve to 1, and the bottom curve to 5. The faint dots indicate where our `pointsArray` coordinates are positioned and are shown for illustrative purposes only; you wouldn't see these rendered normally.

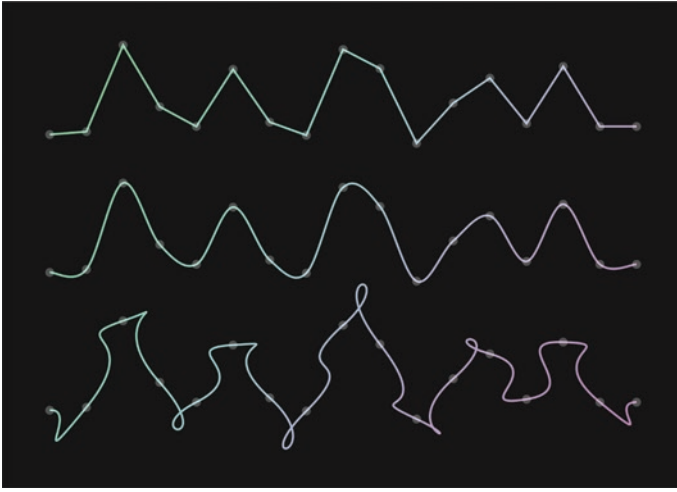


Figure 6-13. *Varying the curve factor using values of 0, 1, and 5*

Summary

With paths now a part of your generative arsenal, so many more possibilities open up for you. To recap, we've covered the following:

- The path data (d) string format and the flexibility and economies it offers.
- The difference between absolute and relative path commands
- The L, V, and H straight-line commands
- The Q and T quadratic Bezier curve commands.
- What control points are and how to harness them
- The elliptical arc curve command A and its complex array of arguments

CHAPTER 6 THE ALL-POWERFUL PATH

- The C and S cubic Bezier curve commands
- Using noise and randomness to shape curves
- The SvJs `createCurve()` method

In the next chapter, we'll explore motion, animation, and how to make our sketches interactive.

CHAPTER 7

Motion and Interactivity

One characteristic that tends to distinguish generative art from other art forms is its ability to incorporate interactivity, and, as a result, active audience participation. This audience – whether it be a single user on a laptop or a small crowd at an art installation – can then influence, or even determine, the end result. In essence, such art is *dynamic*; it moves; it responds; it flows. It isn't fixed and final.

In this chapter, we'll explore some of the underlying techniques of interactive art, including event listeners, callback functions, cursor tracking, collision detection, and how to use these to alter our SVG elements and attributes in real time. We'll also delve into the different methods that can be used to animate SVG and discuss their strengths and weaknesses before deciding on the method we'll adopt for the remainder of the chapter.

Event Listeners

A mouse click is an event, as is hitting a key on your keyboard, or tapping your tablet's touchscreen display. When events like this occur in the context of a web browser, as JavaScript programmers, we can prick up our virtual ears and listen out for them. And as soon as an event is detected, we can capture it. What we do once we've captured an event is up to us.

We could use it to trigger any number of things – a custom message, a color change, a simple animation, a complex transformation; the possibilities are endless. But first we need to know how to “listen.”

Thankfully this is straightforward: we can utilize a browser-native function called `addEventListener()`. This function needs three things: one, an element to attach itself to (i.e., the element that will do the listening); two, the event type to listen out for (e.g., a mouse click); and three, what to do when it hears it (i.e., the action performed by the callback function). It takes the following form:

```
// Adding an event listener to a HTML or SVG element.
element.addEventListener('eventType', doSomething);
```

We are assuming here that the callback function `doSomething()` is defined elsewhere in the code. Note that in this case it is supplied by name only, without the parentheses. This is so that the function doesn’t trigger prematurely. The situation is different with anonymous – or arrow – functions, which can be defined there and then without the danger of being immediately invoked.

```
// Using arrow syntax to define the callback function.
element.addEventListener('eventType', () => {
  // do something here...
});
```

Event Types

There are numerous types of events we can choose to listen out for, but we will focus on a small subset useful for generative art: keyboard, form, and pointer event types.

Keyboard event types like `keydown` and `keyup` capture the different phases of hitting a key as you type. Form event types like `change` and `input` can capture updates to interface elements for inputting data, such

as sliders, text fields, and drop-downs. And pointer event types, perhaps the most significant of the lot where generative art is concerned, are a more modern alternative to mouse and touchscreen event types that aim to be more hardware-agnostic. Pointer event types include `pointerdown`, `pointermove`, `pointerover`, and `pointerup`, and for our purposes, we can also consider the `click` event type – technically a mouse event – as falling under this umbrella, as it can also capture simple touch events like a finger tap.

```
// Capturing the movement of a mouse, stylus or finger over an
// element.
element.addEventListener('pointermove', () => {
  // do something here...
});
```

Event Parameters

When an event listener is attached to an element and captures an event, that event can then be passed to the callback function as an argument. For example, if we were to add an event listener to our HTML document `.body` to detect click events, the properties of that click event would also be captured and could be put to use by the callback function.

Here's how this could work in practice. In the following code, the position of the mouse or pointer device is logged to the console each time you click anywhere within the body of the HTML page.

```
// Log the user's pointer co-ordinates on click.
document.body.addEventListener('click', (event) => {
  let x = event.clientX;
  let y = event.clientY;
  console.log(`x: ${x}, y: ${y}`);
});
```


The `clientX` and `clientY` properties are just a couple of the many properties built into a `click` event. The full list can be found over at MDN (developer.mozilla.org/docs/Web/API/Element/click_event), including comprehensive documentation covering the full range of DOM events.

Triggering the SvJs Save Method

Say we have a sketch we're happy with, and we wanted to save the actual SVG markup generated by our JavaScript – how would we go about it? Perhaps the quickest way is to open up our browser's developer tools, select the Elements tab, and inspect the DOM tree until we locate the root SVG element. Then we can right-click it, select “Copy”, and choose the nested “Copy outerHTML” option. This gives us the SVG file data that we want, but not the file itself, so as a final step, we need to open up a text or code editor, paste our clipboard contents into it, and *then* save it as an SVG file. Not exactly convenient.

It would be far easier to just hit a key on our keyboard and have the process happen automatically. SvJs does have a `save()` method that can be used for this purpose, but it first needs to be “wired up” to an event listener in order to fire. Let's do this now.

Open up any one of our previous sketches, and at the bottom of the sketch, let's write a function to call this method whenever we hit the “s” key. The event type will be `keydown`, which is triggered when any key is pressed, so we need to make sure we pass the event as an argument to our callback function so that we can query its `key` property; this is what tells us the actual key that was pressed. We'll trigger the `save()` method only when that key is equal to “s”, irrespective of case (allowing us to capture the capital “S” too).

```
// Save the root svg as a downloadable file.
document.addEventListener('keydown', (event) => {
  let key = event.key.toLowerCase();
  if (key === 's') svg.save();
});
```

Feel free to use the aforementioned for any sketch going forward. And incidentally, the `save()` method can be called on any SVG element, meaning you can save just a portion of your SVG if you prefer – though the use cases for this are rarer.

Creative Cursor Tracking

In our first example of an event listener in action, we showed how tracking the mouse cursor might be achieved. We simply had to query the `clientX` and `clientY` coordinates of the mouse event. This was purposely simplified, as accurately tracking the position of our mouse relative to the SVG viewport (which uses its own coordinate space) is a bit trickier.

The `SvJs trackCursor()` Method

To solve this, `SvJs` has a built-in `trackCursor()` method that can be called on the root SVG element. It automatically takes care of attaching the relevant event listeners (`pointermove` and `pointerleave`), transforming the cursor position into the `viewBox` coordinate space, and ensuring the expected behavior also occurs with touchscreen devices. To activate it, it just needs to be called on the main SVG element as follows:

```
svg.trackCursor();
```

Once activated, it updates two properties of the parent SVG: `cursorX` and `cursorY`, which tell us where in the `viewBox` (rather than the web browser window) our cursor is currently located. It also allows us to pass in our own custom callback function, to be triggered each time the cursor position is updated.

```
// Passing in a custom callback.
svg.trackCursor(() => {
  // ... do stuff
});
```

Interactive Ellipses

Let's set up a new sketch to show how `trackCursor()` can be put to creative use. Copy our template folder and call it something like `17-cursor-tracking`. We're going to re-create a variation of our very first sketch from Chapter 1, but this time we'll make it interactive.

Below the background, initialize the following variables. The `ellipses` array is a new addition; we'll use this to store the created ellipses so we can iterate through them afterward.

```
// Randomise some variables.
let hue = Gen.random(0, 360);
let rotation = Gen.random(0, 360);
let iterations = Gen.random(50, 100);

// This array will allow us to iterate through our
ellipses later.
let ellipses = [];
```

Now for the loop to create the ellipses. The code here is more compact than that presented in the first chapter, as we've since covered more concise ways of achieving certain results (like incrementing our hue). Another difference of note is the addition of the `ellipses.push()` method; this is where we add each ellipse to the aforementioned array.

```
// Run a loop a random number of times to create the ellipses.
for (let i = 0; i < iterations; i += 1) {

  // Create our ellipse.
  let ellipse = svg.create('ellipse');
  ellipse.set({
    cx: 500,
    cy: 500,
    rx: 100 + (i * 3),
```

```

    ry: 300 + (i * 2),
    fill: 'none',
    stroke: `hsl(${hue} 80% 80% / 0.6)`,
    transform: `rotate(${rotation + (i * 2)} 500 500)`
  });

// Add the ellipse to the array.
ellipses.push(ellipse);

// Increment the hue.
hue = (hue % 360) + 2;
}

```

You should now see a static arrangement of ellipses. Our next step is make them dynamic. What we'll do is activate the `trackCursor()` method on the main SVG and pass in an arrow function as a callback. Inside this arrow function, we'll iterate through each item in the `ellipses` array using a `forEach()` loop and adjust the `cx` and `cy` coordinates of each ellipse relative to our cursor position.

```

// Adjust the centre point of each ellipse relative to
our cursor.
svg.trackCursor(() => {
  ellipses.forEach((ellipse) => {
    ellipse.set({
      cx: svg.cursorX,
      cy: svg.cursorY
    });
  });
});

```

And with this, we have movement – our ellipses glide gracefully into life as we create a truly dynamic cursor-guided composition! Figure 7-1 illustrates one possible variation with the cursor just south of center. A key-mapped save function is particularly useful here (I used it myself to save down Figure 7-1), as it allows us to capture our SVG without disturbing the composition with inadvertent cursor movement (as might happen if we had to click a button).

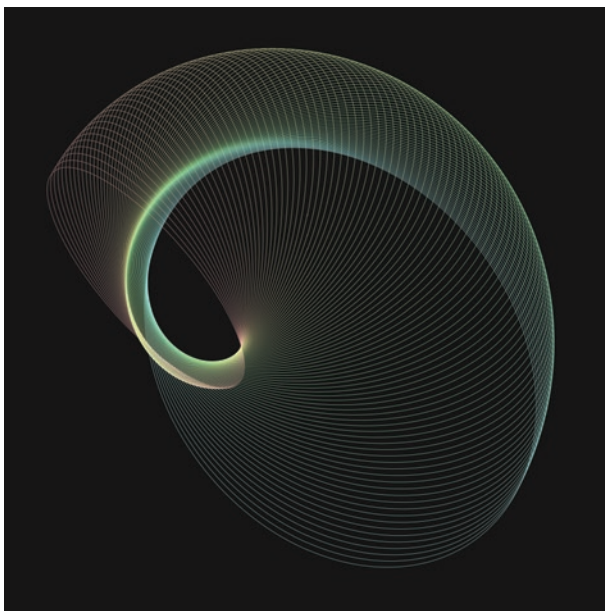


Figure 7-1. *A cursor-transformed array of ellipses*

Now that we know how to achieve movement via interactivity, it's time to learn how to make things move independently of our input.

Programming Motion

As mentioned previously, we have several different methods at our disposal for animating SVG. The first one we'll explore is CSS keyframes, which will be familiar territory to most web developers. The second is

the method native to the SVG format itself – SMIL. The third is the Web Animations API (or WAAPI), which is a modern JavaScript-driven API. And the final method is a technique popular among generative artists (of the JavaScript variety at least): the recursive `requestAnimationFrame` method.

Knowledge of each technique and their respective strengths and weaknesses should serve you well, irrespective of which one we proceed with. We'll set up a simple sketch that will showcase, in a very rudimentary way, all four options. When you're ready, set up a new instance of our template sketch and call it something like 18-making-things-move.

What we're going to do is create a diamond shape comprised of four overlaid squares of different dimensions (as shown in Figure 7-2) and animate each one using a different technique. The following code sets things up, and apart from the `transform_origin` attribute (which will ensure more consistent animation behavior across the various methods by setting a default point of rotation), there's nothing really new here, so feel free to simply copy and paste:

```
import { SvJs } from '../..node_modules/svjs/src/index.js';

// Parent SVG.
const svg = new SvJs().addTo(document.
getElementById('container'));

// Viewport and viewBox (1:1 aspect ratio).
const svgSize = Math.min(window.innerWidth, window.
innerHeight);
svg.set({ width: svgSize, height: svgSize, viewBox: '0 0 1000
1000' });

// Background.
svg.create('rect').set({
  x: 0, y: 0, width: 1000, height: 1000, fill: '#181818'
});
```

```
// Arrays to contain our shapes and their colours.
let palette = ['#34d399', '#6ee7b7', '#a7f3d0', '#d1fae5'];
let shapes = [];

// Initialise our four shapes.
for (let i = 0; i < 4; i += 1) {
  let size = 500 - (i * 125);
  let position = 250 + (i * 62.5);
  let shape = svg.create('rect').set({
    x: position,
    y: position,
    width: size,
    height: size,
    fill: palette[i],
    transform_origin: '50% 50%',
    transform: 'rotate(45)'
  });
  shapes.push(shape);
}
```



Figure 7-2. The four shapes we'll make move

CSS Keyframes

Animated sequences can be created with CSS using a `@keyframes` rule. Much like traditional keyframes used in animation, motion graphics, and video editing software, CSS `@keyframes` define the state of certain properties at a particular point in time (or step in a sequence). For example, at time t , our subject might be at position $[x1, y1]$, and at time $t + 5$, the subject might have moved to $[x2, y2]$. Keyframes define these points in time, and the transition between one state and another is what constitutes the animation. We call this process “tweening” (meaning “inbetweening”), as the animation occurs in these inbetween states.

The syntax for CSS `@keyframes` is quite self-explanatory. You can set the start and end keyframes of a simple animation with `from` and `to` directives, or if you have something a little more complex in mind, you can use percentages to specify the point at which certain transitions should

trigger. Once the `@keyframes` rule is defined, it then needs to be called by an element using the `animation-name` property, and we also need to specify how long the animation should take (the `animation-duration`). There are other optional properties that we can define, such as the `animation-timing-function` (also known as easing), which describes *how* we transition from one property value to another, and the `animation-iteration-count`, which specifies how many times our animation should play.

In the following code, we're calling the first shape of our array and assigning it an id of `cssShape`. This will enable us to easily target the element later. We then set up a `@keyframes` rule called `scaleRotate` that rotates its target a full 360 degrees, while simultaneously scaling it down along its x axis and up along its y axis (creating a stretching effect) before returning it to its original dimensions. And finally we assign the animation to the `cssShape` element, specifying the duration, iteration count, and timing function.

```
// Set an id for our first shape.
shapes[0].set({ id: 'cssShape' });

// Animate this shape with CSS.
svg.create('style').content(`
  @keyframes scaleRotate {
    0% { transform: rotate(0) scale(1, 1) }
    50% { transform: rotate(180deg) scale(0.5, 1.5) }
    100% { transform: rotate(360deg) scale(1, 1) }
  }

  #cssShape {
    animation-name: scaleRotate;
    animation-duration: 5s;
    animation-iteration-count: infinite;
    animation-timing-function: linear;
  }
`);
```

You should now see our outer square slowly rotate and stretch, while the others remain static. It's also possible to use the shorthand `animation` property, which allows for greater concision by declaring all our animation values at once. Only use this when familiar with the longhand version though, as the order of values – while not especially strict – can lead to unanticipated behavior if you're not familiar with all the possible values that can be accepted.

```
svg.create('style').content(`
  ...

  #cssShape {
    animation: scaleRotate 5s infinite linear;
  }
`);
```

The SMIL Way

Now let's try to achieve the same thing with SMIL. SMIL, which stands for Synchronized Multimedia Integration Language, is the animation method native to SVG itself, requiring no CSS or JavaScript. However, as we're using SvJs, we will be interfacing with SMIL via JavaScript. Before we do so however, here's a quick peek at how it looks in its original form.

```
<circle cx="50" cy="50" r="25" fill="red">
  <animate
    attributeName="r"
    from="25"
    to="40"
    dur="2s"
    repeatCount="indefinite" />
</circle>
```

As you can see, the animation is an actual SVG element, nested inside the element it animates (a `<circle>` in this case). The `<animate>` element takes care of the animation of most SVG attributes, but in the case of element transforms (i.e., translating, rotating, scaling, or skewing), an `<animateTransform>` element must be used instead. There is one other animation element called `<animateMotion>`, reserved for animating an element along a path. As we will be rotating and scaling our shape, the `<animateTransform>` element is the one we'll be using.

Each transform requires its own element, making the syntax a little verbose. With the first transform, namely, the rotation, we specify the `attributeName` to animate and its type, `rotate`. We have just two states to transition between – 0 degrees and 360 degrees – so we can use `from` and `to` directives to cover these. We then set the duration (`dur`) and iteration count (`repeatCount`), and that's the first transform taken care of.

```
// Rotate the second shape using SMIL.
shapes[1].create('animateTransform').set({
  attributeName: 'transform',
  type: 'rotate',
  from: '0',
  to: '360',
  dur: '5s',
  repeatCount: 'indefinite'
});
```

The second transform is a little more complicated. It has more than two states, so instead of the `from` and `to` attributes, we have a `values` attribute that can store multiple states. In this case, we supply it with a semicolon-separated list of our transform values. We also need to include an additional `additive` attribute with a value of `sum` so that the scale transform adds to (and doesn't overwrite) the previous rotate transform.

```
// Scale the second shape using SMIL.
shapes[1].create('animateTransform').set({
  attributeName: 'transform',
  type: 'scale',
  values: '1 1; 0.5 1.5; 1 1',
  dur: '5s', additive: 'sum',
  repeatCount: 'indefinite'
});
```

The second shape should now be slowly rotating and stretching, and the motion should be perfectly synchronized with our CSS keyframes example.

The Web Animations API

The Web Animations API, or WAAP, is the most modern of the methods we'll be covering. It is being actively worked on by the W3C (World Wide Web Consortium) and is intended to provide the future direction for animation on the Web. In addition to keyframes and timing options, it supports advanced features such as control over playback rates, flexible easing functionality, scroll-linked timelines, and the ability to apply successive animations (or sequences) to the same element.

Elements can be animated with a sensibly named `animate()` method, which takes two arguments. The first argument defines the keyframes, and the second defines the timing options.

```
element.animate(keyframes, options);
```

The keyframes can be supplied as either an array of objects or an object containing arrays. And the timing options can be supplied as a single value – specifically, an integer defining the length of the animation

in milliseconds – or as an object containing one or more time-related properties. Say we wanted to animate the value of a circle’s radius r . The following is how this could be achieved using both syntax formats:

```
// Format one for keyframes.
let keyframes = [
  { r: 25 },
  { r: 40 },
  { r: 25 }
];

// Format two for keyframes.
let keyframes = {
  r: [25, 40, 25]
};

// Format one for timing options.
let options = 2000; // 2 seconds.

// Format two for timing options.
let options = {
  duration: 2000,
  iterations: Infinity,
  // ... etc
};
```

I prefer the second keyframe format as it is more concise in most cases. Before we implement the rotation and scaling example, there’s one more thing to point out: how the keyframes are spaced out (or offset) from one another. With CSS keyframes, we need to explicitly state when a keyframe should trigger by using percentage values or the `from` and `to` directives. With the Web Animations API, the offset of animations is *inferred* if not explicitly stated. If we have three keyframes, they will be automatically

spaced out at 0, 0.5, and 1, respectively (the equivalent of 0%, 50%, and 100% in CSS). But we can define custom offsets if needs be. In the following example, the second keyframe will trigger at 0.75 instead of the default 0.5.

```
// Explicitly defining offsets.
let keyframes = {
  r: [25, 40, 25],
  offset: [0, 0.75, 1]
};
```

Let's get back to our sketch now and implement the rotation and scaling of our third shape using the WAAPI method. Below our SMIL implementation, set up the animation as follows.

```
let keyframes = {
  transform: [
    'rotate(0deg) scale(1, 1)',
    'rotate(180deg) scale(0.5, 1.5)',
    'rotate(360deg) scale(1, 1)'
  ]
};

let options = {
  duration: 5000,
  iterations: Infinity
};

shapes[2].animate(keyframes, options);
```

Now all but one of our shapes should be on the move. Note how similar the WAAPI keyframes syntax is to our CSS keyframes example; this isn't by accident. The Web Animations API is essentially a further development of and extension to preexisting web animation standards so that web developers familiar with CSS animation should find the syntax intuitive

and (mostly) compatible with what they already know. Having a native, high-level JavaScript API to create animations in the browser has opened up possibilities previously only attainable through the use of third-party libraries, but it isn't the only game in town where native JavaScript animation techniques are concerned.

The `requestAnimationFrame()` Method

Meet the `requestAnimationFrame()` method, a low-level recursive technique that allows us to tap into the browser's screen repainting process. This is a very different approach from the methods we've covered previously; it is at the same time very simple and very complex. By calling a single method over and over again (this is the recursive part), we are handed full, fine-grained control over the entire animation process – but this also means we have to do everything ourselves. Concepts like keyframes and timing options don't really apply at this level.

The `requestAnimationFrame()` method must be called within a custom function we define, which contains the code we want to run on a per-frame basis. We then pass this custom function as an argument to `requestAnimationFrame`, so it is called recursively on each screen repaint. Here's a basic example of how this works.

```
// Define the function.
function customFunction() {

    // do stuff here...

    // Make the recursive call to requestAnimationFrame().
    requestAnimationFrame(customFunction);
}

// Call the function.
customFunction();
```

Additionally, `requestAnimationFrame()` can pass a timestamp to our custom function so that we can sync our animation with real-time values rather than relying on the screen refresh rate, which can vary from device to device. The timestamp contains the amount of milliseconds that have elapsed since our animation started. Let's continue now with our sketch and set up an `animate()` function and pass it the timestamp so that we can use it to increment the angle of rotation. This works because rotation values can increment indefinitely beyond 360° and still remain valid.

```
// Animate the last shape using requestAnimationFrame.
function animate(time) {

  // Rotate the angle.
  let angle = time;

  // Apply the rotation.
  shapes[3].set({ transform: `rotate(${angle})` });

  // The recursive bit.
  requestAnimationFrame(animate);
}

animate();
```

Ok, so this code gets our final shape moving, but we've got a couple of problems. One, our shape is rotating at a breakneck speed, wildly out of sync with the others. And two, if you inspect the console, you'll see an error to the effect that we're supplying an undefined variable to our rotation transform. This error occurs just the once, so what that tells us is that when our function runs for the first time, the `angle` (and therefore the `time`) is undefined. This is simple enough to rectify; we just need to reassign the `time` variable to a value of 0 if it is undefined. Include the following code before the `angle` is declared, and that should soothe our console's concerns:

```
// Prevent errors when the time is undefined on first frame.
if (time === undefined) time = 0;
```


Now let's address the speed of rotation. We could try dividing the time by arbitrary amounts until it slows down to the point of synchronization (or something close enough), but that's not very satisfactory. Instead, let's work with what we know:

- Our shape should rotate by 360°.
- This rotation should take five seconds (or five thousand milliseconds).
- The time is provided in milliseconds.

Based on the aforementioned, we can derive a calculation that gives us the exact speed by which to rotate our shape: 0.072° per millisecond (360° divided by 5,000 milliseconds). Update the previous statement where angle simply equals time with the following:

```
// Rotate 360° in 5000ms: 360/5000 = 0.072.
let angle = time * 0.072;
```

This should rotate our shape in sync with the others. Getting it to scale in sync, however, is more of a challenge. For that, we first need to initialize some variables outside of the animation loop that can help us keep track of certain states. Why? Well, if we tried creating these states inside the loop, they would get overwritten on each iteration.

The scale increment value needs to swing back and forth between an upper and a lower limit, and the most straightforward way to achieve this is to keep track of the current scale value and its polarity (i.e., whether it's swinging one way or another). For this, we'll need a boolean that we can toggle back and forth like a switch. We'll also need a way to increment the scale value in a way that synchronizes with the time but doesn't use the timestamp value directly as this would only increment it indefinitely.

With the aforementioned in mind, let's initialize the following variables before the `animate()` function:

```
let isPositive = true;
let scale = 0, tick = 0, prevTime = 0;
```

Update the animation loop as follows. The comments do the explanatory heavy-lifting here, so to better understand each step, be sure to read the comments carefully. The main takeaway here is just how much more work we need to do with `requestAnimationFrame()` compared to the other methods.

```
// Animate the final shape using requestAnimationFrame.
function animate(time) {

  // Prevent errors when the time is undefined on first frame.
  if (time === undefined) time = 0;

  // Rotate 360° in 5000ms: 360/5000 = 0.072.
  let angle = time * 0.072;

  // We need a constant tick value that doesn't increment
  indefinitely.
  tick = time - prevTime;

  // Scale by 0.5 in 2500ms: 0.5/2500 = 0.0002.
  scale = isPositive ? scale + (tick * 0.0002) : scale -
    (tick * 0.0002);

  // Apply the rotation and scale values.
  shapes[3].set({
    transform: `rotate(${angle}) scale(${1 - scale},
      ${1 + scale})`
  });
});
```

```

// Flip the polarity if the scale value falls outside
// these bounds.
if (scale < 0 || scale > 0.5) isPositive = !isPositive;
// Capture the time before it increments.
prevTime = time;

// The recursive bit.
requestAnimationFrame(animate);
}

animate();

```

We should now have the final shape rotating and scaling in sync (give or take) with the other shapes. In some cases, you might see some slight offset in the rotation, or the scaling might happen a little quicker than expected. Discrepancies like this have more to do with how the browser automatically schedules and handles optimizations with the other animation methods than it has to do with the performance of `requestAnimationFrame()` itself. These animation methods weren't designed to be combined as we've done in this sketch; normally you'll choose one method and stick to it, depending on what you're trying to achieve.

Methods Summarized

So which method should you choose for generative art? There's no definitive answer to this, but for what it's worth, here's my advice (with the caveat that this is not a comprehensive analysis of the pros and cons of each method, but a cursory comparison from my inevitably partisan point of view).

CSS keyframes are more suited to shorter sequences, such as animated loading indicators (an art in themselves), UX enhancements, and animations where accessibility or browser compatibility is a concern. For more elaborate sequences, they can quickly get convoluted and awkward to edit. For animating SVG specifically, there are many attributes that are inaccessible when using CSS, such as masks, clip paths, and gradients.

With SMIL, you've got a well-thought-out animation format native to SVG and one that can access all animation-capable attributes. Unlike the other methods, it can also retain its animations when the SVG is embedded as an image (much like a gif). The negatives? It can become quite verbose, and its future is rather unclear (back in 2015, support for SMIL was dropped from Chrome, but this decision was quickly reversed).

The Web Animations API, or WAAP, is the most modern animation method, has more advanced capabilities than CSS or SMIL, and in general is more flexible and concise. I would recommend it over CSS or SMIL for generative art, but it does have its limitations. It's not suitable for open-ended, exploratory animations, where we're unsure of the end result. Like CSS and SMIL, everything needs to be tightly orchestrated and intentional. To go "full generative," we need something more low level like `requestAnimationFrame()`.

Yes, `requestAnimationFrame()` can make us trudge a rather punishing path to achieve relatively meager results, but conversely, more complicated results can in many cases be achieved in a far more simple and elegant manner than they can using other methods (I'll demonstrate this in our next example on collision detection). We just need to persevere beyond the initial pain barrier.

It's also worth noting that there are many solid third-party libraries out there to help you animate with SVG; of particular note is the GreenSock Animation Platform (GSAP), which is very feature rich and offers robust performance and impressive backward compatibility. It's not open source however, and some of its plug-ins are for paid subscribers only. Anime.js is a nice open source alternative.

Collision Detection

It's one thing to interact ourselves with a generative piece to influence the output, but it's another thing entirely when it begins to interact with *itself*. This is in essence what collision detection entails. We'll explore an example using the bounds of a rectangle as a basic collision barrier, and we'll utilize the `requestAnimationFrame()` method while it's still relatively fresh in our minds.

Setting Boundaries

So the usual drill – create a new folder from the template, rename it to 19-collision-detection, and begin coding below the background. The first thing we'll do is create a rect styled like a picture frame. This effect is achieved by using a slightly transparent stroke over the fill; half the stroke is painted outside the frame and half inside (something we'll need to factor into our collision detection calculations later). The size of this frame will be randomized, so we'll set up a `frameSize` variable to capture this and also use it to keep other element (like the stroke-width) in proportion.

```
// Create a frame to act as the boundary.
let frameSize = Gen.random(350, 700);
let frame = svg.create('rect').set({
  x: (1000 - frameSize) / 2,
  y: (1000 - frameSize) / 2,
  width: frameSize,
  height: frameSize,
  fill: '#252525',
  stroke: `hsl(${Gen.random(0, 360)} 80% 80% / 0.25)`,
  stroke_width: frameSize / 10,
});
```

Initializing and Extending Our Shapes

With the frame now in place, let's fill it with some circles of various sizes. The aim will be to have these circles burst out from the center in all directions, some rushing rapidly and others ambling slowly, and all of them bouncing off the frame borders as they encounter them. In the following, we'll start by initializing the circles and filling up an array as we've done in our previous examples.

```
// Randomise the number of circles and set up an empty array.
let numCircles = Math.floor(frameSize / 20);
let circles = [];

// Populate this array.
for (let i = 0; i < numCircles; i += 1) {

  // Randomise the radius relative to the frame size.
  let radius = Gen.random(frameSize / 100, frameSize / 25);

  // Apply the above variables and randomise the hue.
  let circle = svg.create('circle').set({
    cx: 500,
    cy: 500,
    r: radius,
    fill: `hsl(${Gen.random(0, 360)} 80% 80% / 0.5)`
  });

  // Store the circle in the array.
  circles.push(circle);
}
```

We have our circles all clustered in the center now, but they're not yet primed for movement. We have a way to track their position (via the `cx` and `cy` attributes), but we have no attributes to track their speed or direction, which we'll need if we're going to pull off collision detection.

Thankfully SvJs elements are easily extendable. If we want to add a new attribute to an element, it's as simple as including it in the `set()` method. If it's not a valid SVG attribute, it won't have any impact on the rendered SVG, but that's ok – we simply want it stored to be used in our collision detection calculations later.

We'll need at least two new attributes: one to track the speed on the x axis and another to track the speed on the y axis. To determine direction, we can use these same attributes and invert the polarity to change the direction. For example, positive numbers on the x axis will indicate left-to-right motion, and negative numbers will mean right-to-left motion. On the y axis, positive numbers will denote downward motion and negative numbers upward motion.

Setting these new attributes will be a two-step process. We'll first set up variables `velocityX` and `velocityY` to capture randomized speeds. And the attributes we'll set will be simply `vx` and `vy`, and we'll randomize their polarity using `Gen.chance()` (so they'll have a 50/50 chance of being either positive or negative). Update the relevant code as follows:

```
// Create variables to control the speed on the x and y axes.
let velocityX = Gen.random(0.1, 5, true);
let velocityY = Gen.random(0.1, 5, true);

// Apply the above variables and randomise the hue.
let circle = svg.create('circle').set({
  cx: 500,
  cy: 500,
  r: radius,
  vx: Gen.chance() ? velocityX : -velocityX,
  vy: Gen.chance() ? velocityY : -velocityY,
  fill: `hsl(${Gen.random(0, 360)} 80% 80% / 0.5)`
});
```

Frame-by-Frame Calculations

Calculating whether or not one of our circles collides with a boundary (i.e., the edge of our frame) involves keeping a constant check on each circle's position. For this reason, `requestAnimationFrame()` is really the only feasible way to simulate collision detection, as these checks need to occur on a frame-by-frame basis.

There's a bit of a code dump coming up, but I'll explain the important parts. You'll notice that when we fetch any of our attributes, we're wrapping them in a `Number()` method. This is because our attributes will be returned as strings, and if we're to do any reliable calculations with them, we want to explicitly handle them as numeric values rather than relying on JavaScript's covert casting (which may sometimes get things right, but it leaves you wide open to bugs).

Here's a rundown of the algorithm we'll implement. First, we calculate the lower and upper bounds (i.e., the frame edges). Then, for each circle:

- We retrieve the position and velocity and store them as variables.
- If the x variable is less than or equal to either the lower bound or the upper bound, we reverse the polarity of the x velocity variable.
- We repeat the previous step for the y axis.
- We then update the x and y variables based on the calculated velocity.
- Finally, we apply the updated variables to the circle element.

Here's the concrete coded version:

```
// Get the lower and upper bounds (the frame edges) for use in
the loop.
let lowerBound = Number(frame.get('x'));
let upperBound = lowerBound + frameSize;

// The animation loop.
function animate() {

  // Check collisions for each circle.
  circles.forEach((circle) => {

    // Retrieve the position and velocity.
    let cx = Number(circle.get('cx'));
    let cy = Number(circle.get('cy'));
    let vx = Number(circle.get('vx'));
    let vy = Number(circle.get('vy'));

    // Check for collisions, and if found reverse the polarity.
    if (cx <= lowerBound || cx >= upperBound) vx = -vx;
    if (cy <= lowerBound || cy >= upperBound) vy = -vy;

    // Update the position.
    cx += vx;
    cy += vy;

    // Set the new values.
    circle.set({ cx: cx, cy: cy, vx: vx, vy: vy });
  });

  // The recursive bit.
  requestAnimationFrame(animate);
}

// Call the animation.
animate();
```

If you watch the animation at this point, you'll notice that the circles do indeed rebound within the vicinity of the frame edge, but it's far from precise and even appears to differ depending on the circle size. Looks like we've left something out of our calculations.

To fix this, we need to account for the fact that the frame border is inset; that is, the stroke is halfway inside the frame, and the radius of each circle is different. This means that the lower bound and upper bound variables must be calculated *inside* the animation loop, as they will vary per circle. There are, however, a couple of static values that can be captured outside the loop and used later inside it: the frame edge and frame inset. Delete the lower bound and upper bound variables and replace them with the following:

```
// Get the frame edge point (x or y) and the inset.
let frameEdge = Number(frame.get('x'));
let frameInset = Number(frame.get('stroke-width')) / 2;
```

Now, inside the loop, we can work out the proper lower bound and upper bound values.

```
// The animation loop.
function animate() {

  // Check collisions for each circle.
  circles.forEach((circle) => {

    // Calculate the lower and upper bounds for each circle.
    let radius = Number(circle.get('r'));
    let lowerBound = frameEdge + radius + frameInset;
    let upperBound = frameEdge + frameSize - radius -
      frameInset;

    // ... remainder of code is unchanged.

  })
}
```

This should result in accurate collision detection; Figure 7-3 shows a freeze-frame of this in action. There are a lot of ways you could take this further – add noise into the mix, see if you can incorporate cursor tracking, or change the colors over time. As it is however, we’ve taken the first steps on the road to full-on physics simulations (a topic way beyond the scope of this book, but I’d encourage you to explore it independently).

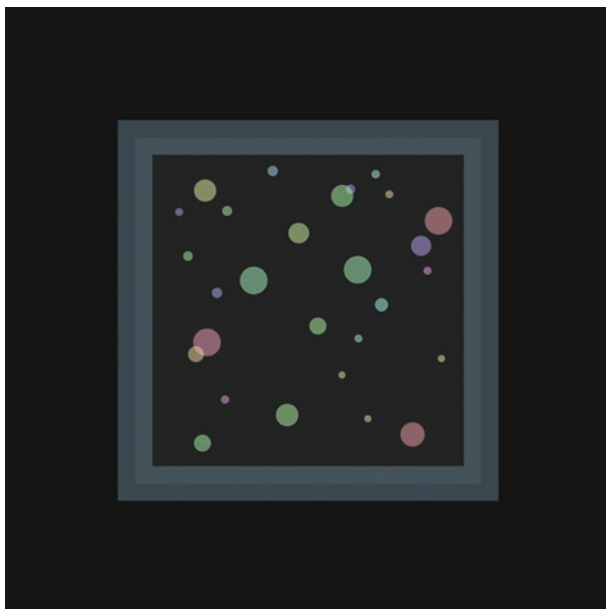


Figure 7-3. *Colorful collision detection*

Circularity

We’ll round off the chapter with a return to the JavaScript Web Animations API. We’ll touch on trigonometry in the process, but only tangentially (if you got that pun, you’re doing well!), as the main focus will be to provide you with some inspiration, not laden you with complicated math.

Let's create a new example folder called 20-circular-loop, copying our template folder. The aim will be to create a looping animation consisting of circles that are themselves arranged in a circle; these kinds of animations hold high hypnotic potential. We'll start in the usual place below our background, initializing some randomized variables.

```
// Randomise some variables.
let numCircles = Gen.random(20, 35);
let baseRadius = Gen.random(5, 25, true);
let hue = Gen.random(0, 360);
```

We'll use numCircles to control how many times our loop runs and also in the calculations needed to distribute these circles. The baseRadius and hue we'll use as starter values. For now, create the loop and the circles as follows, omitting the cx and cy attributes (which we'll work out in the next section).

```
// Set up the loop.
for (let i = 0; i < numCircles; i += 1) {
  // Create the circle, but don't set the position or
  // radius yet.
  let circle = svg.create('circle').set({
    r: baseRadius,
    fill: 'none',
    stroke: `hsl(${hue} 80% 80% / 0.75)`,
    transform_origin: '500 500'
  });
}
```

Slices of PI

Here comes the trigonometry tangent we mentioned earlier. To calculate the distribution of each of our circles (i.e., their *cx* and *cy* coordinates), we need to draw upon some built-in JavaScript Math methods and properties. The first is `Math.PI`, a property that returns the approximated value of π (3.14), a mathematical constant that represents the ratio of a circle's circumference (the length along its perimeter) to its diameter (double the radius). Figure 7-4 illustrates these values.

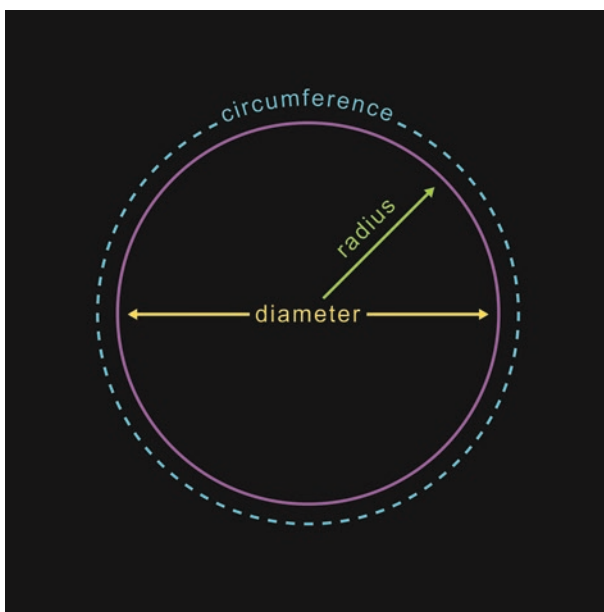


Figure 7-4. *The circumference, diameter, and radius of a circle*

You may be accustomed to angles being expressed in terms of degrees, but in JavaScript, angles are measured in radians by default. One radian is equivalent to the length of a circle's radius wrapped around the circumference (visualized in Figure 7-5). It takes just over six of these radians (or approximately 6.28) to wrap around the full circumference. And half of 6.28 is π , or 3.14.

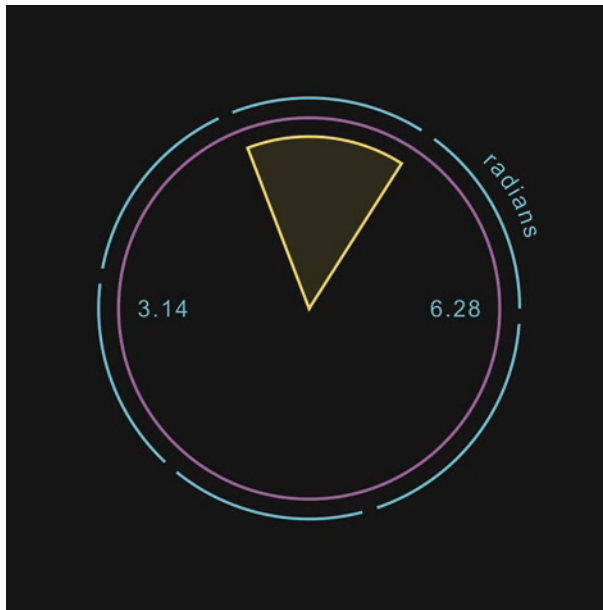


Figure 7-5. *How radians relate to π and the circumference*

The takeaway? `Math.PI radians` is equivalent to 180° , meaning `Math.PI * 2` gives us a full circle (360°). So within the context of our loop, we're going to increment the angle at which our circles are placed relative to `Math.PI * 2`. And to ensure an equal distribution of our circles along the circumference, we'll divide `Math.PI * 2` by `numCircles`, and each time the loop runs, we'll multiply it by our iterator to move on to the next "slice," so to speak.

Within the loop, after we define our circle element, include the following line:

```
// Calculate the current angle.
let angle = Math.PI * 2 / numCircles * i;
```

Sine and Cosine

Once we have the angle, we need to get the corresponding sine and cosine values, which you can think of as the x and y coordinates. We won't go into too much detail here; suffice to say, the methods `Math.sin()` and `Math.cos()` work together perfectly to create circular motion.

```
// Get the sine and cosine of the angle.
let sin = Math.sin(angle);
let cos = Math.cos(angle);
```

It's also worth noting that these are extremely useful methods in their own right, particularly for motion, due to the smooth, oscillating values they produce. For the current sketch, however, we'll limit ourselves to using these values to position our circles only, not animate them. And to position them, we need to map the sine and cosine values to a more usable range, as they return values between -1 and 1 by default.

We'll position our circles relative to the center of the `viewBox` (so 500 on both the x and y axes) and re-map them so that they span a range of 700. This gives us 150 and 850, that is, -350 and +350 either side of the center.

```
// Map the sine and cosine to the desired range.
let cx = Gen.map(sin, -1, 1, 200, 800, false);
let cy = Gen.map(cos, -1, 1, 200, 800, false);
```

Animating Our Circles

We can now try our hand at a basic animation. We'll start simple and move our circles outward from the center toward the `cx` and `cy` coordinates we defined earlier.

```
// Move from (500, 500) to (cx, cy).
circle.animate({
```

```

    cx: [500, cx, 500],
    cy: [500, cy, 500],
  }, {
    duration: 10000,
    iterations: Infinity,
    easing: ['ease-in-out']
  });

```

This gets things moving, but it's not an especially interesting sequence. Let's add more variety to it by setting an initial and target radius, using the iterator value to vary these per circle, and rotating the entire animation while we're at it. And just before we close out the loop, let's increment the hue relative to the number of circles to get a nice color distribution.

```

// Set the initial and target radii.
let r1 = baseRadius * 2 + (i * 10);
let r2 = baseRadius / (i + 10);

// Move from (500, 500) to (cx, cy), reduce the radius, rotate.
circle.animate({
  cx: [500, cx, 500],
  cy: [500, cy, 500],
  r: [r1, r2, r1],
  transform: ['rotate(0deg)', 'rotate(360deg)']
}, {
  duration: 10000,
  iterations: Infinity,
  easing: ['ease-in-out']
});

// Increment the hue.
hue = (hue % 360) + (180 / numCircles);

```


There... That gets us much closer to achieving that hypnotic pull. See Figure 7-6 for a snapshot of this animation at about 15% progression.

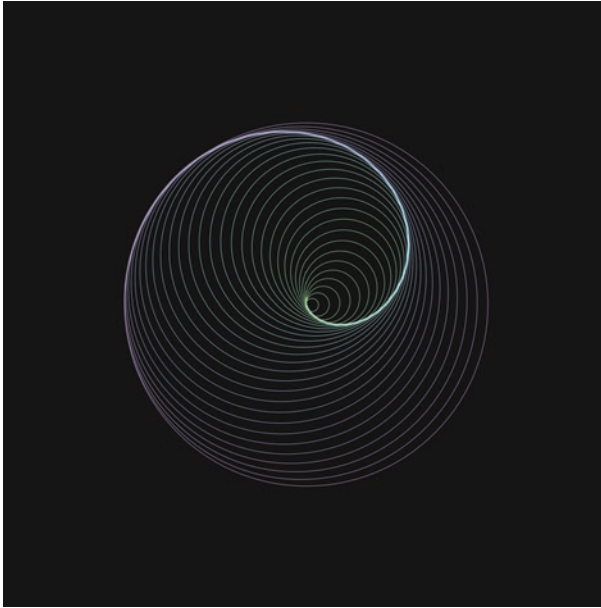


Figure 7-6. *A circular loop with the Web Animations API*

Summary

This has been perhaps the densest of all chapters so far in terms of concepts and content, but I hope you’ve also had some fun following along. To quickly recap, we’ve covered the following:

- Event listeners, event types, and event parameters
- How to save our SVG markup
- How to use cursor tracking to make a sketch interactive
- Animating with CSS keyframes
- Animating with SMIL

- The Web Animations API
- Animating with the `requestAnimationFrame()` method
- Basic collision detection
- Very basic trigonometry

In the next and final full chapter, we'll focus on SVG filters.

CHAPTER 8

Filter Effects

In addition to all the vector functionality offered by SVG (as you'd expect from a format called Scalable Vector Graphics), SVG also provides some very powerful pixel-based effects in the form of the `<filter>` element. Many of the kinds of image-manipulation operations routinely found in editors like GIMP or Photoshop, such as blurs, drop shadows, and distortions, can actually be carried out in SVG too. In this chapter, we'll learn how this is done.

We won't be covering the capabilities of SVG filters in any kind of comprehensive way; that would require at least its own book. It is a vast subject and one whose creative potential – at least aesthetically – is often neglected. It's very easy to make awful-looking SVG filters. I'll aim to avoid this pitfall and will be presenting a subset of filters more from the perspective of the generative artist than the image editor.

Filter Fundamentals

Before we explore specific filter effects in any depth, we need to know how they're structured. A filter consists of a single `<filter>` element and one or more filter primitives. These primitives, which are also elements, are nested within the main filter element, and they all begin with the letters `fe` (for filter effect) followed by a descriptive name. Each performs its own operation(s) and can be connected to other primitives in a node-like manner using a series of inputs and outputs.

Let's look at a basic example in raw markup form, using just a single primitive.

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0
1000 1000">
  <defs>
    <filter id="blur">
      <feGaussianBlur stdDeviation="10" />
    </filter>
  </defs>
  <circle r="350" cx="500" cy="500" fill="yellow"
filter="url(#blur)" />
</svg>
```

As you can see, the `<filter>` element must be nested inside a `<defs>` element; in common with gradients and patterns, it only gets rendered when referenced via a `url` link to its `id`. In the preceding case, a yellow circle calls the filter via its own `filter` attribute, and the result (shown in [Figure 8-1](#)) is a soft blur, controlled by a primitive named `feGaussianBlur`.



Figure 8-1. *A blurred circle*

The Ins and Outs

Let's inspect a slightly more involved example, this time illustrating how filter primitives can be connected together. The result will be a drop shadow applied to a circle's stroke, as shown in Figure 8-2.

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0
1000 1000">
  <defs>
    <filter id="shadow">
      <feGaussianBlur stdDeviation="25" in="SourceAlpha"
        result="blur" />
      <feOffset dx="0" dy="50" in="blur" result="offset" />
      <feComposite in="SourceGraphic" in2="offset" />
    </filter>
  </defs>
```

```
<circle r="300" cx="500" cy="500" fill="none"  
stroke="#475569" stroke-width="50" filter="url(#shadow)" />  
</svg>
```



Figure 8-2. A drop-shadow filter effect

As with the previous example, we're starting off with an `feGaussianBlur` primitive, but you'll notice there's a couple of more attributes added: `in` and `result`. These attributes are also present in the primitive following it, `feOffset`. In the final filter primitive, `feComposite`, `in` is also present, along with `in2`. What do these mean?

- `in` refers to a filter primitive's input source.
- `in2` refers to a second input source, if the filter primitive supports this.
- `result` is what we use to label a filter primitive's output, which can then be used as the input for another primitive.

If a filter primitive is the first in its chain (as `feGaussianBlur` is above), `in` will default to `SourceGraphic`; otherwise, it will default to the result of the preceding primitive. `SourceGraphic` just means the graphics element to which a filter is applied. `SourceAlpha` also refers to this same graphics element but is restricted to its alpha channel (i.e., its opacity value).

So here's what's happening in this second example:

1. The `feGaussianBlur` primitive takes the `SourceAlpha` of the circle as its input and applies a Gaussian blur effect to it. The result (or output) is then named `blur`.
2. The `feOffset` primitive takes the `blur` result as its input and offsets it along the y axis, and the result of this is labelled `offset`.
3. The `feComposite` primitive takes our original `SourceGraphic` as the first input `in` and the result of `offset` as its second input `in2`, recombining them.

We'll go into some of these primitives in more detail later, but next we'll look at how to move from markup to scripting filters with SvJs.

The SvJs `createFilter()` Method

Perhaps unsurprisingly, SvJs has its own `createFilter()` method to speed up the filter creation process. It requires just one argument: an `id`, and this `id` is used when applying the filter to the target element. As with the `createGradient()` and `createPattern()` methods, sensible defaults are initialized, and the checking of whether a `defs` element already exists is taken care of under the hood.

The following is the SvJs equivalent of our first example shown previously, applying a soft blur to a yellow circle (Figure 8-1).

```
// Initialise the filter.  
let filter = svg.createFilter('blur');  
  
// Create the blur effect.  
filter.create('feGaussianBlur').set({ stdDeviation: 10 });  
  
// Apply the filter.  
svg.create('circle').set({  
  r: 350, cx: 500, cy: 500, fill: 'yellow', filter:  
  'url(#blur)'  
});
```

The Filter Region

When a filter is applied to an element, its active region extends beyond that element. This allows a certain amount of space into which the effect can “bleed,” to use a print analogy. This space is known as the filter region, and once a filter’s effect extends beyond this region, clipping occurs (see Figure 8-3).

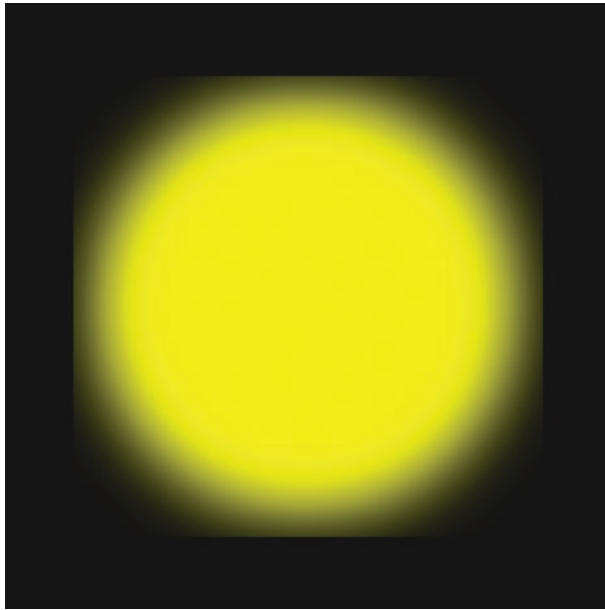


Figure 8-3. *A clipped filter region*

In generative art, we'll sometimes stray outside the range of values a filter effect parameter might anticipate, so it's important to know about this region if your filter effects are unexpectedly clipped. A filter region is defined just as a `rect` element is: by `x`, `y`, `width`, and `height` attributes. By default, these values are as follows:

```
<filter x="-10%" y="-10%" width="120%" height="120%">
```

The percentage values are relative to the shape to which the filter is applied, so in the aforementioned, this translates to a 10% padding surrounding the filtered element. SvJs enlarges this filter region to 25%, allowing some more room to experiment. If you need to adjust it further, you can easily do so with the `set()` method:

```
filter.set({ x: '-15%', y: '-15%', width: '130%', height: '130%' });
```

Filter Effects 101

Let's move on to some simple custom filter effects. We've already seen the `feGaussianBlur` primitive in action so I won't give it its own section here, but it's worth mentioning before I move on that its main attribute `stdDeviation` can actually accept two values as well as one. If you supply two, the first number will control the blur along the x axis and the second the blur along the y axis.

```
// Create a 'motion' blur effect.
filter.create('feGaussianBlur').set({ stdDeviation: '20 0' });
```

Shadows

We implemented a drop shadow earlier using three connected filter primitives, but there's actually a much easier way to do this. In a more recent revision of the SVG spec, the primitive `feDropShadow` was introduced, which can accept up to five attributes: `dx`, which defines the shadow offset along the x axis; `dy`, which does the same along the y axis; `stdDeviation`, which controls the blurring of the shadow; and two more optional attributes, `flood-colour` and `flood-opacity`, which can modify the shadow's color and transparency.

Here's how we could rewrite the code for Figure 8-2 using SvJs and the `feDropShadow` primitive.

```
// Initialise the filter.
let filter = svg.createFilter('shadow');

// Create the drop shadow.
filter.create('feDropShadow').set({ dx: 0, dy: 50,
stdDeviation: 25 });

// Apply the filter.
svg.create('circle').set({
```

```
// ... other attributes.
  filter: 'url(#shadow)'
});
```

Coloring

We mentioned earlier some attributes called flood-colour and flood-opacity that can be used with the `feDropShadow` primitive. There's another primitive these can also be applied to and to which they originally belonged: `feFlood`. The `feFlood` primitive is very basic; it requires no input and simply floods the filter region with a specific color and opacity. By itself, it's not especially interesting, but it can be useful when combined with other primitives.

Other more capable color-related primitives exist called `feColorMatrix` and `feComponentTransfer`. The latter is too advanced to cover in this book so we'll limit ourselves to `feColorMatrix` and `feFlood`. Let's set up a sketch and put these to use.

Instead of copying our template folder, let's copy our first grid example, which I labelled 07-regular-grids (see Chapter 4). Copy and rename this folder to something like 21-colour-filter. Leaving the rest of the code as is, replace the body of the nested for loop with the following:

```
if (Gen.chance(55)) {
  grid.create('rect').set({
    x: x, y: y, width: cellSize, height: cellSize,
    fill: `rgb(
      ${[Gen.random(0, 255), Gen.random(0, 255), Gen.
        random(0, 255)]}
    )`
  });
}
```

This should give us a chance arrangement of colorful tiles on each refresh. So far we've been quite restrained in our use of color, controlling mainly the hue component of the `hsl()` function. This time we're unleashing the full `rgb` color spectrum, so the results can range from dramatic and dynamic to garish and grating (see Figure 8-4 for some sample output). There is something I like about these results; the coloring, however, can get a little too incongruous at times. So let's use some filters to rein in the colors but retain the dynamic range.

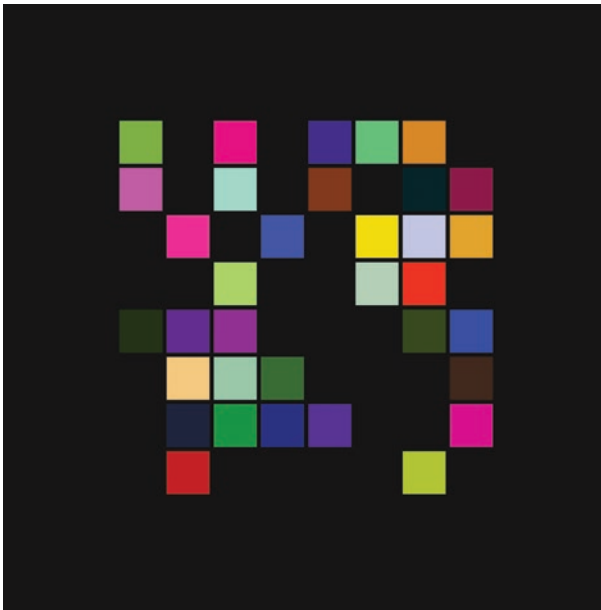


Figure 8-4. *Full-spectrum RGB tiles*

The first step is to desaturate the colors. For this, we can use the `feColorMatrix` primitive, which takes two arguments: `type` and `values`. The accepted values depend on the `type`, which can be as follows:

- `saturate`: This determines the color saturation of the input. It accepts values between 0 (no saturation) and 1 (full saturation), though higher values may be accepted by browsers to allow for over-saturation.

- **hueRotate:** This rotates the hue value by a specified angle, which is set in values. Setting values to 180 for example would invert all the hue values of the input source.
- **luminanceToAlpha:** This converts the lightness of the input to alpha values and requires no values.
- **matrix:** This is the most complex and powerful of the `feColorMatrix` types. It accepts a 5 by 5 color transformation matrix as the values, which matrix-multiplies the pixel component values of the input source. It's a little too advanced to delve into in this chapter.

Of the aforementioned, `saturate` is what we're looking for. To drain the colors from our input, we need to set the values to 0. Let's set up a filter now to do this. At the bottom of the sketch, initialize a new filter as follows:

```
// Initialise the filter.
let filter = svg.createFilter('colourise');

// De-saturate the input.
filter.create('feColorMatrix').set({
  type: 'saturate',
  values: 0,
  result: 'desaturate'
});

// Apply the filter.
grid.set({ filter: 'url(#colourise)' });
```

This should result in a grayscale grid. Next, we want to choose a color to mix into it, and for this, we'll use the `feFlood` primitive. Between the `feColorMatrix` primitive and the last line where we apply the filter, add the following code:

```
// Set a flood colour.  
filter.create('feFlood').set({  
  flood_color: '#7F462C',  
  result: 'flood'  
});
```

This should add a sepia-brown color to the composition. The problem? It simply floods the canvas, covering everything else. We need a way to blend it with the result of the `feColorMatrix` desaturation.

Blending

Most vector and image editing applications have blending capabilities built in, in the form of layer blending modes. SVG supports this feature at the filter level in the form of the `feBlend` primitive.

A blending mode (or just blend mode) defines how colors interact when elements overlap (whether the elements are individual shapes or entire layers). If a blend mode is set to `multiply` for example, the final color is the result of multiplying the top color by the bottom color, which creates an effect similar to transparent film cells overlapping. If set to `screen`, the top and bottom colors are inverted, multiplied, and re-inverted, resulting in an effect similar to two images shone onto a projection screen.

There are a number of blend modes supported by the SVG spec, and describing each one is beyond the scope of this chapter, but for a comprehensive treatment (including interactive examples), check out the MDN entry at developer.mozilla.org/docs/Web/CSS/blend-mode.

There are actually several blend modes that will work well in our case, so what I'm going to do is store these in an array so we can select from them later. Include the following code before our filter is initialized:

```
// Set up a blend modes array.
let blendModes = ['screen', 'overlay', 'lighten', 'color-
dodge', 'soft-light', 'color'];
```

Next, after where we defined the `feFlood` primitive, set up the `feBlend` primitive. We'll randomize its mode attribute and set the first input to the flood color and the second input to the desaturated grid.

```
// Randomise the blend mode.
filter.create('feBlend').set({
  mode: Gen.random(blendModes),
  in: 'flood',
  in2: 'desaturate',
  result: 'blend'
});
```

The first input, `in`, acts as our top color, and the second input, `in2`, acts as our background color. Our grid should have re-appeared at this point in colorized form, but we've still got our flood color in the foreground. We need a way to strip this out, which brings us to the next section.

Compositing

If you've ever watched the behind-the-scenes footage of some of your favorite films, you'll likely have come across the green-screen technique: the action hero who scales a skyscraper is, in reality, hanging from a pulley with green fabric draped in the background. This technique makes it easier to strip away the background from the foreground and combine it later with separately captured footage.

Compositing is the act of fusing these two disparate elements together; two sources combine and become one. It is similar to blending in many respects, but the results are more pronounced and programmable. It has more to do with how elements intersect than how their colors merge, and the various compositing operations available to us can produce very different results. Blending, on the other hand, tends to be more uniform in its output.

The `feComposite` primitive allows us to perform compositing operations in SVG. It takes two inputs, `in` and `in2`, and there are seven possible values its operator attribute accepts: `over`, `in`, `atop`, `xor`, `out`, `lighter`, and `arithmetic`. In Figure 8-5, I've illustrated six of these; the `arithmetic` operator has no set output (it's fully customizable) and is a little too complex to properly cover in this chapter.

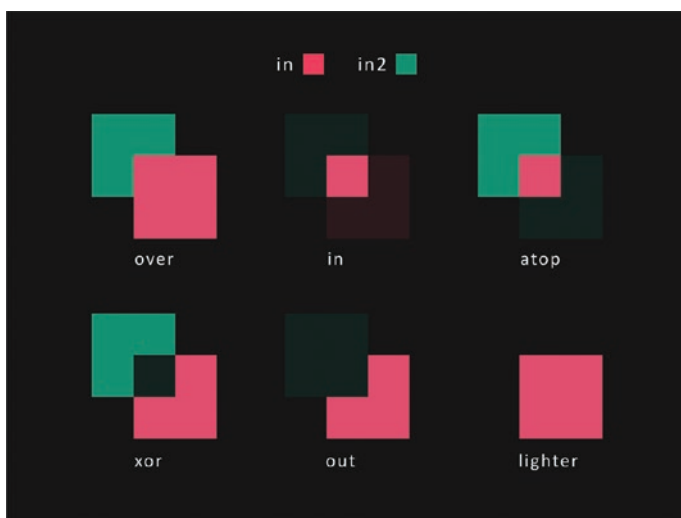


Figure 8-5. Six of the seven SVG compositing operations

Because the `feComposite` primitive is so useful and something you'll likely lean on if you further explore SVG filters, it's worth describing each preset operator. In Figure 8-5, we can think of `in` as our foreground and `in2` as our green-screen background.

- **over:** This is the default operator, where our foreground is simply placed over the background.
- **in:** The portion of the foreground that overlaps with the background becomes the new output.
- **atop:** The portion of the foreground that overlaps with the background is retained; the rest of the foreground is clipped.
- **xor:** Those portions of the foreground and background that do not overlap are kept; the rest is removed. It's essentially the inverse of the **in** operator.
- **out:** The portion of the foreground that falls outside the background is displayed.
- **lighter:** The overlapping regions of the foreground and background are summed together (resulting in a lightening effect).
- **arithmetic:** If this operator is chosen, four additional attributes must be defined, all numeric: k_1 , k_2 , k_3 , and k_4 . The output is then defined via the following formula: $(k_1 * i_1 * i_2) + (k_2 * i_1) + (k_3 * i_2) + k_4$, where i_1 and i_2 represent **in** and **in2**, respectively. Yep, told you it was complex! But the most subtle and customizable results can be achieved using this operator, so it is well worth independent exploration.

If we take the result of the **feFlood** primitive as our first input (the foreground) and the **SourceGraphic** as the second input, the operator that will achieve the effect we're looking for is **atop**. To recap, with this operator, those portions of the foreground (**feFlood**) that overlap with the background (**SourceGraphic**) are retained; the rest of **feFlood** should be clipped.

Let's set this up now. Include the following code after the `feBlend` primitive, before the filter is applied to the grid.

```
// Composite the blend 'atop' the original.
filter.create('feComposite').set({
  in: 'blend',
  in2: 'SourceGraphic',
  operator: 'atop'
});
```

For me, this resulted in the output you see in Figure 8-6. I don't know about you, but I see a pugnacious pup putting up his dukes, à la Scrappy-Doo (though that may well be my pareidolia at work). On each refresh, you'll see something quite different, and while the color tones vary, a common palette can be discerned due to the single `feFlood` color we're drawing upon. This kind of effect would be quite difficult to achieve without filters.



Figure 8-6. *The result of the composite operation*

Noise and Distortion

In this section, we'll introduce a new kind of noise, one that's actually built in to the SVG spec itself. And we'll also explore how to add some distortion to our filter pipeline. With these two ingredients, along with the basics covered earlier, a whole world of effects opens up to us.

Turbulence

In Chapter 5, we introduced the SvJs Noise module, which allowed us to use Perlin noise in our sketches. SVG also implements Perlin noise inside its `feTurbulence` filter primitive, so what's the difference?

Well, with the Noise module, we can extract and play with noise *data*, using its values to shape our vectors. With `feTurbulence` on the other hand, we have a filter primitive that renders noise data directly, in *pixel* form. This doesn't give us access to the underlying noise data, but what we can do is easily alter the visual output by tweaking its attributes and then feed it into other filter primitives. The `feTurbulence` attributes are as follows:

- `baseFrequency`: This is the only mandatory attribute, and we can think of it as defining the noise “zoom level.” It's typical for the lowest value to start at 0.0001 (though you can start lower), and the higher the values go, the more of our noise terrain is revealed. If you supply two numeric values instead of one, the first will define the frequency along the x axis and the second along the y axis. Decoupling the values this way makes it possible to stretch or compress the resulting texture horizontally or vertically.

- `numOctaves`: This attribute refers to the number of octaves present in the noise signal and determines the level of detail we can discern. Higher values render more natural results but are more processor-intensive. The most usable results lie in the range of 1 to 5.
- `seed`: This allows us to change the “location” from which we sample our noise. If the `baseFrequency` is our zoom level, the seed is our position. A valid seed value should be a positive integer anywhere from 0 to 10 million.
- `stitchTiles`: This attribute accepts either `stitch` or `noStitch` and refers to how adjacent areas of the noise terrain are blended together. With it set to `noStitch` (the default), certain noise configurations can lead to a “tiling” effect, where the boundary between noise regions is quite pronounced. Setting a value of `stitch` can soften this.
- `type`: This can be set to `fractalNoise` or `turbulence` (the default). The former produces the same kind of noise pattern across the red, green, blue, and alpha channels, and the latter produces a different pattern in the alpha channel. The `fractalNoise` type has a softer look and is great for generating gaseous textures like clouds. The `turbulence` type, on the other hand, is more liquid-like in appearance.

In Figure 8-7, we can see two instances of `turbulence` on the left and two instances of `fractalNoise` on the right. The values vary only slightly between instances, which should give you a sense of the potential variation offered by this filter primitive.